

System Inspekcji Butelek – Instrukcja

Uruchomienie modelu yolo i algorytmów klasycznych na danych testowych:

Z poziomu folderu TWM_PROJEKT26L:

predykcja modelu yolo:

```
python implementation\run_pipeline_yolo.py --step predict
```

detekcja braku zakrętki:

```
python implementation\classic\cap\run_cap_test.py
```

detekcja zanieczyszczeń:

```
python implementation\classic\debris\run_debris_test.py
```

detekcja etykiety:

```
python implementation\classic\label\run_label_test.py
```

Opis modułów

common/bottle_detector.py

Wspólny moduł wykorzystywany przez algorytmy klasyczne.

Zawiera funkcję:

```
get_bottle_bbox(...)
```

Funkcja odczytuje bounding box butelki z pliku etykiet YOLO i zwraca:

```
(bx, by, bw, bh)
```

Dzięki temu wszystkie algorytmy mogą pracować na obszarze rzeczywistości zawierającym

butelkę.

common/evaluate.py

Uniwersalny moduł ewaluacji dla wszystkich klasycznych detektorów.

Generuje:

- Accuracy
 - Precision
 - Recall
 - F1-score
 - Confusion Matrix
 - `misclassified.csv`
-

cap/detect_cap.py

Klasyczna detekcja obecności zakrętki.

Wykorzystuje:

- Wyznaczenie ROI w górnej części butelki.
- Analiza koloru zakrętki.
- Obliczenie udziału pikseli odpowiadających kolorowi zakrętki.
- Porównanie z ustalonym progiem.

Klasy:

- `good`
 - `no_cap`
-

debris/detect_debris.py

Klasyczna detekcja zanieczyszczeń.

Wykorzystuje:

- Wyznaczenie ROI wewnątrz butelki.
- Konwersja do przestrzeni HSV.
- Segmentacja pikseli odpowiadających zanieczyszczeniom.
- Obliczenie procentu zajętej powierzchni.
- Porównanie z progiem.

Klasy:

- `good`
 - `debris`
-

label/detect_damaged_label.py

Klasyczna detekcja uszkodzonej lub brakującej etykiety.

Wykorzystuje:

- Wyznaczenie ROI w miejscu występowania etykiety.
- Analiza kolorów charakterystycznych dla etykiety.
- Obliczenie udziału pikseli należących do etykiety.
- Porównanie z ustalonym progiem.

Klasy:

- `good`
 - `damaged_label`
-

yolo/yolo_train.py

1. Wczytanie obrazów i anotacji YOLO.
 2. Przekazanie obrazów do sieci YOLO.
 3. Predykcja klas i bounding boxów.
 4. Porównanie predykcji z anotacjami.
 5. Obliczenie funkcji strat (box loss, cls loss, dfl loss).
 6. Aktualizacja wag modelu metodą backpropagation.
 7. Powtarzanie procesu przez kolejne epoki.
 8. Walidacja modelu po każdej epoce.
 9. Zapis modelu o najlepszych wynikach walidacyjnych ([best.pt](#)).
-

yolo/yolo_predict.py

1. Wczytanie obrazu.
 2. Przeskalowanie do rozmiaru wejściowego modelu.
 3. Przepuszczenie obrazu przez wytrenowaną sieć YOLO.
 4. Wykrycie obiektów (bounding box).
 5. Klasyfikacja wykrytego obiektu do jednej z klas:
 - good
 - wrong_bottle
 - underfilled
 - no_cap
 - loose_cap
 - debris
 - damaged_label
 6. Zwrot klasy, współczynnika pewności oraz bbox.
-

Dane testowe

Obrazy:

```
dataset_yolo/test/images
```

Etykiety:

```
dataset_yolo/test/labels
```

Wyniki

Wszystkie wyniki klasycznych metod zapisywane są do:

```
outputs/classical_results/
```

Przykładowe katalogi:

```
outputs/classical_results/cap/  
outputs/classical_results/debris/  
outputs/classical_results/label/
```

Każdy moduł generuje:

- `annotated/` – obrazy z wizualizacją detekcji
- `*_results.csv` – szczegółowe wyniki
- `confusion_matrix.png`
- `metrics.txt`
- `misclassified.csv`

Wszystkie wyniki z yolo zapisywane są do:

```
outputs/yolo_predictions/
```

Klasy YOLO

ID	Klasa
0	good
3	no_cap
5	debris
6	damaged_label

Pozostałe klasy zgodnie z aktualną konfiguracją datasetu.

YOLO

Trenowanie

```
python yolo/yolo_train.py
```

Predykcja

```
python yolo/yolo_predict.py
```

Domyślnie wykorzystywany model:

```
yolo26n.pt
```

Uwagi

- Obecnie klasyczne algorytmy cap i label wykorzystują bbox butelki zapisany w etykietach

YOLO.

- Jeżeli bbox nie zostanie znaleziony, algorytmy przechodzą na domyślnie zdefiniowany ROI.
- Przed uruchomieniem należy upewnić się, że dla każdego obrazu istnieje odpowiadający mu plik `.txt`.
- Wyniki ewaluacji są generowane automatycznie po zakończeniu testu.